COMP 364/464: High-Performance Computing

Week 03, Lecture 03

Agenda

1. Networks
    a. Theoretical terms and applications
    b. Types
    c. Real machines
2. Runtime
    a. Nodes
    b. Linux
    c. Schedulers, tie-back to networks
3. HW 2 and Reading 2 announcement

Course Schedule

Wk	Date	Topic	Readings	HW	Projects
1	Aug 26	Intro to the class + planning questionnaire	—	—	—
2	Sep 2	Intro to HPC: architectures, networks, topics overview	—	—	Project 1 assigned; Project 3 announced
3	Sep 9	Networks + architectures (cont.) + Linux, finalize	R1	HW1	
4	Sep 16	Parallel programming (part 1)	R2	HW2	Project 1 due; Project 2A assigned
5	Sep 23	Parallel programming (part 2)	R3	HW3	
6	Sep 30	Performance monitoring	R4	HW4	Project 2A due, 2B assigned; Project 3 topic selection due
7	Oct 7	Performance monitoring + midterm review	R5	HW5	
8	**Oct 14**	**Midterm**	—	—	Project 3 draft survey due
9	Oct 21	Virtualization in HPC	R6	HW6	Project 2B due
10	Oct 28	Data in HPC + Security	R7	HW7	
11	Nov 4	Fault tolerance	R8	HW8	
12	Nov 11	AI in HPC	R9	HW9	
13	Nov 18	Open — possible guest speaker	R10	HW10	Project work time
14	Nov 25	Thanksgiving break	R11 (opt)	—	
15	Dec 2	Wrap-up / presentations + final review	—	—	Project 3 all parts due

How does an HPC system work?

1. Get a really massive machine
2. Give it a huge, high-performance network
3. Get a specialized OS that keeps things safe, sane, and timely, plus a "super-OS" to schedule on it
4. Make a job that can run in parallel
5. Schedule it, requesting whatever resources it needs
6. The scheduler finds a time when the machine has those resources, then sets it to run
7. Job starts, and the nodes do their business
8. Job completes, and the user finds their resulting data

How does an HPC system work?

1. **Get a really massive machine**
2. Give it a huge, high-performance network
3. Get a specialized OS that keeps things safe, sane, and timely, plus a "super-OS" to schedule on it
4. Make a job that can run in parallel
5. Schedule it, requesting whatever resources it needs
6. The scheduler finds a time when the machine has those resources, then sets it to run
7. Job starts, and the nodes do their business
8. Job completes, and the user finds their resulting data

How does an HPC system work?

1. **Get a really massive machine**
2. **Give it a huge, high-performance network**
3. Get a specialized OS that keeps things safe, sane, and timely, plus a "super-OS" to schedule on it
4. Make a job that can run in parallel
5. Schedule it, requesting whatever resources it needs
6. The scheduler finds a time when the machine has those resources, then sets it to run
7. Job starts, and the nodes do their business
8. Job completes, and the user finds their resulting data

How does an HPC system work?

1. **Get a really massive machine**
2. **Give it a huge, high-performance network**
3. **Get a specialized OS that keeps things safe, sane, and timely, plus a "super-OS" to schedule on it**
4. Make a job that can run in parallel
5. Schedule it, requesting whatever resources it needs
6. The scheduler finds a time when the machine has those resources, then sets it to run
7. Job starts, and the nodes do their business
8. Job completes, and the user finds their resulting data

Networks: What is it?

- The differentiating factor in what makes an HPC system more than just a large-scale machine
- How two nodes communicate
- The incredibly expensive second computer that sits alongside the compute nodes
- Spaghetti mess of cables somehow tamed through the sheer will of a lot of network engineers
- Miles and miles (and miles) of copper and glass
- The only reason nodes have storage, the internet, maintenance, etc

Networks: What's in it?

For being so complex, they're made of incredibly simple ingredients.

Like a souffle, but binary.

- NICs
- Switches
- Cables (optical, copper)

Networks: How do they work?

Now that's a bit harder…

- Usually multiple networks, actually
  - High-speed peer-to-peer network for compute
  - Lower-speed maintenance network, monitoring stack comms, scheduler access, etc
  - ??? speed interconnect to the storage framework
  - A subset of nodes will also have access to the good ol' internet
- Static or dynamic
- "Smart" or "dumb"
- Protocol varies, typically a variant on ethernet these days
- How they're connected varies enormously -- think edges of geometric shapes

Networks: Vocabulary

- Static
- Dynamic
- Degree
- Diameter
- Arc connectivity
- Bisection width
- Cost (link or switch)

Networks: Vocabulary

- **Static**
- Dynamic
- Degree
- Diameter
- Arc connectivity
- Bisection width
- Cost (link or switch)

- Network that does not route
- Like water along canals
- “Packet goes into network, packet comes out”
- Can include dumb switches
- Pros:
  - Easy to design
  - Easy to implement
  - Easy to assess
- Cons:
  - Scaling is impossible

Networks: Vocabulary

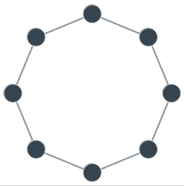
- Static
- **Dynamic**
- Degree
- Diameter
- Arc connectivity
- Bisection width
- Cost (link or switch)

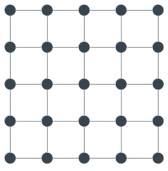
- Network that does route
- Like cars along roads
- “Packet goes into network, packet makes any number of turns and reroutes, then packet comes out”
- Nodes and switches can act as smart switches
- Pros:
  - Scalable
  - Intricate
- Cons:
  - Expensive - switches, design, debugging, all of it

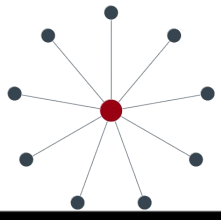
Networks: Vocabulary

- Static
- Dynamic
- **Degree**
- Diameter
- Arc connectivity
- Bisection width
- Cost (link or switch)

- Actually a descriptor of a node or switch:
  - Number of links attached to a node/switch
- Fixed: Doesn't change, even if the network scales
- Uniform: The same across the entire machine
  - This is…actually rare
- We will generally discuss "max degree" of nodes
- Gives an idea for how many wires come out of a component - a short-hand for "complexity"







Networks: Vocabulary

- Static
- Dynamic
- Degree
- **Diameter**
- Arc connectivity
- Bisection width
- Cost (link or switch)

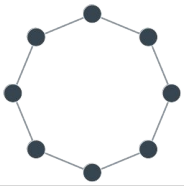
- Describes the number of "hops" to get from one node to another
- We will use both min and average
- Short-hand for performance
- Note: fence post problem!
  - How do we count it?

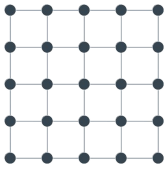
4 nodes, 3 hops for the diameter

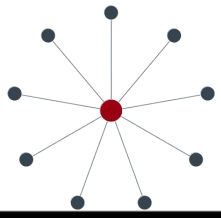
Networks: Vocabulary

- Static
- Dynamic
- Degree
- Diameter
- **Arc connectivity**
- Bisection width
- Cost (link or switch)

- Number of links to slice in order to break the network into two distinct parts
- Short-hand for robustness
- Lots of classes spend time calculating this --- we won't, but I will show you the numbers



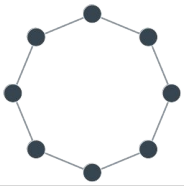


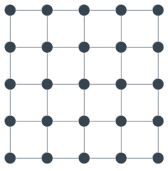


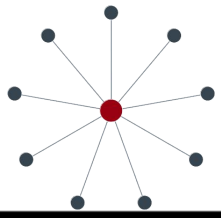
Networks: Vocabulary

- Static
- Dynamic
- Degree
- Diameter
- Arc connectivity
- **Bisection width**
- Cost (link or switch)

- Number of links to slice in order to break the network into two distinct ***halves***
- Not really a commonly-used term
- We actually use this to discuss bandwidth
  - Multiply by per-link BW to get **bisection bandwidth**
- Now *that* gets used. Short-hand for network performance.
- Again, lots of classes work on calculating this - we won't

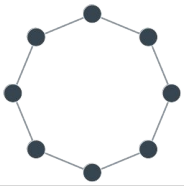


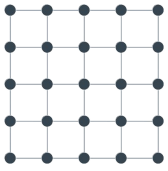


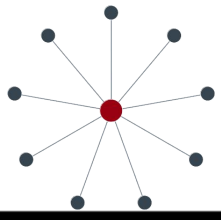


Networks: Vocabulary

- Static
- Dynamic
- Degree
- Diameter
- Arc connectivity
- Bisection width
- **Cost (link or switch)**
  - Number of lines (links) or points (switches or nodes)
  - Short-hand for actual cost of the network
    - Human effort
    - Organization madness
    - Cap-ex $$$





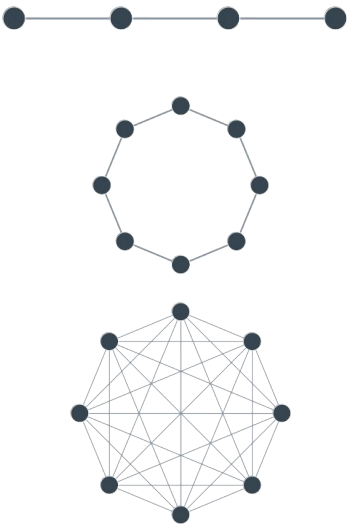


Networks: Vocabulary Short-Hands

- Static: Simple network that won't scale
- Dynamic: Potentially *very* complicated network that will
- Degree: Maximum, how complicated is this network?
- Diameter: Minimum/Average hops, how performant is it?
- Arc connectivity: How easy is it to break it?
- Bisection width: How much bandwidth can it handle?
- Cost (link or switch): How many human headaches and $$ will this cost?

Networks: Point-to-Point

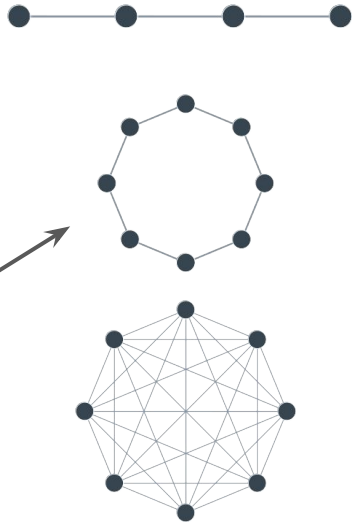
- Two nodes connected via a link
- Ex: linear array, ring, fully-connected
- Pros:
  - Easy (static)
  - Potentially very performant
- Cons:
  - Scaling (...static)
  - Cost (human and $$)



Degree:	2
Diameter:	n-1
Arc Connectivity:	1
Bisection width:	1
Cost (links):	n-1

Networks: Point-to-Point

- Two nodes connected via a link
- Ex: linear array, ring, fully-connected
- Pros:
  - Easy (static)
  - Potentially very performant
- Cons:
  - Scaling (...static)
  - Cost (human and $$)

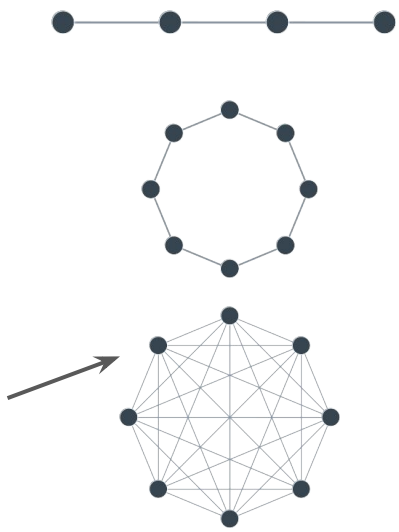


Degree:	2
Diameter:	⌊n/2⌋
Arc Connectivity:	2
Bisection width:	2
Cost (links):	n

Networks: Point-to-Point

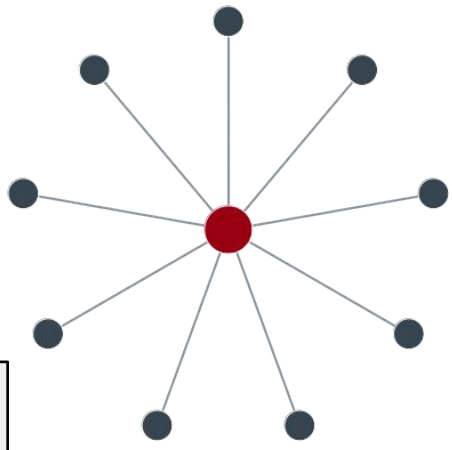
- Two nodes connected via a link
- Ex: linear array, ring, fully-connected
- Pros:
  - Easy (static)
  - Potentially very performant
- Cons:
  - Scaling (...static)
  - Cost (human and $$)

Degree:	n-1
Diameter:	1
Arc Connectivity:	n-1
Bisection width:	$(n / 2)^2$
Cost (links):	n(n-1) / 2



Networks: Star

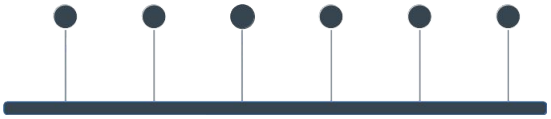
- Central “hub”
- Pros:
  - Resilient (other than the hub)
  - Simple, mirrors a lot of algorithms
- Cons:
  - Scaling and cost are hard (bigger hub?)
  - Making the hub resilient takes work



Degree:	p-1
Diameter:	2
Arc Connectivity:	1
Bisection width:	1 (remember, this is BW shorthand)
Cost (links):	p-1

Networks: Bus

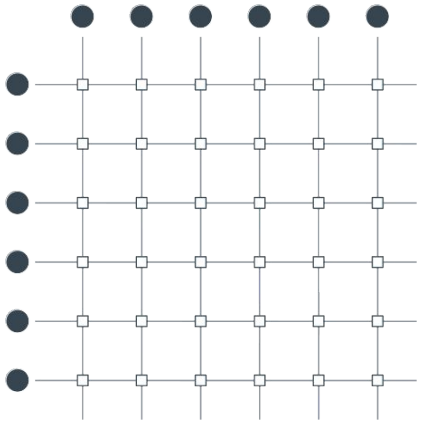
- All nodes connected along wide, shared cable
- Pros:
  - Easy (static)
  - Potentially very performant
- Cons:
  - The large shared cable becomes the cost
- Now the backplane on a node board



Degree:	1 (but the bus is thick)
Diameter:	1
Arc Connectivity:	1
Bisection width:	1
Cost (links):	??? (depends on how many cables in the bus)

Networks: Crossbar

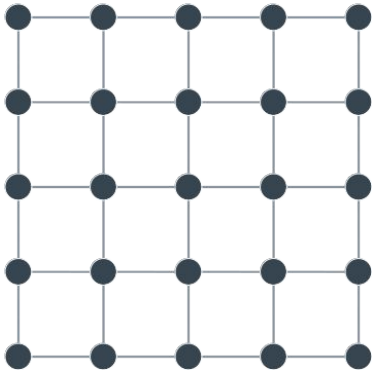
- Non-blocking any-to-any
- Pros:
  - Ideal!
  - Mirrors many computation steps
- Cons:
  - Scaling is incredibly expensive
- Note: this is now the switch!



Degree:	1
Diameter:	1…ish
Arc Connectivity:	1
Bisection width:	p / 2
Cost (links):	p*((p/2) - 1) + p
Cost (switches):	$(p/2)^2$

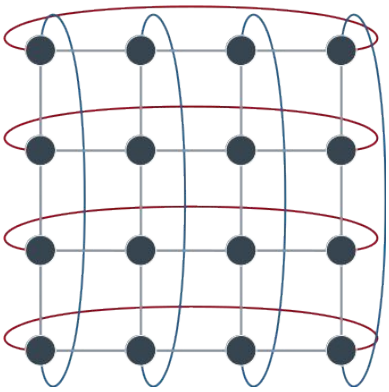
Networks: Mesh

- Non-uniform
- Pros:
  - Great for neighbor computation
  - Predictable, logical
- Cons:
  - Scaling works poorly
  - Non-uniformity makes for placement headaches
- Immediate improvement: wrap



Degree:	4 (max, or 3 or 2)
Diameter:	2 * (√p - 1)
Arc Connectivity:	2
Bisection width:	√p or √p + 1
Cost (links):	2 * √p * (√p - 1)

Networks: 2D Torus



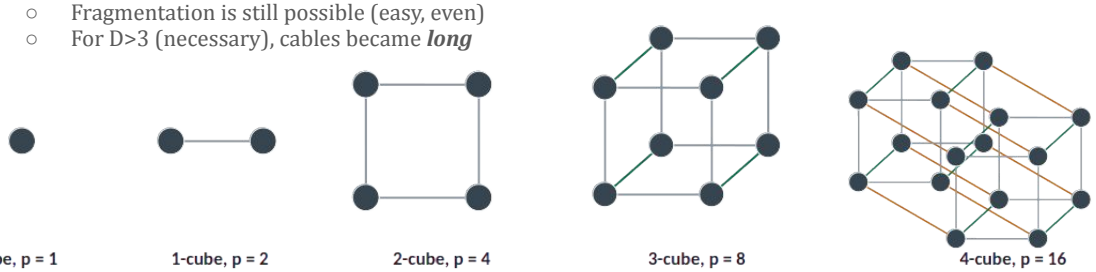
- Every row/column makes a ring
- Pros:
  - Uniform! Easy!
  - Neighbor-neighbor comms are simple
- Cons:
  - Fragmentation is still possible (easy, even)
- Immediate improvement: more dimensions!

Degree:	4
Diameter:	$2 * \lfloor \sqrt{p} / 2 \rfloor$
Arc Connectivity:	4
Bisection width:	$2\sqrt{p}$
Cost (links):	$2p$

Networks: Hypercube (because dimensions are hard)

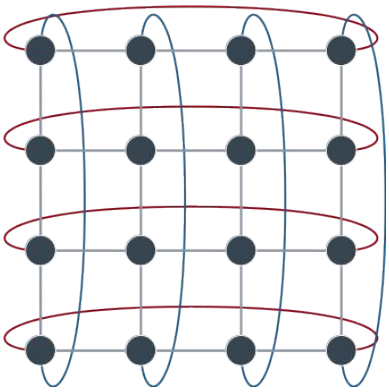
- k-ary D-cube
- These are more theoretical - wires got *long*
- Pros:
  - Easy theoretical math!
  - *Really* easy routing - XOR the labels, flip one bit at a time
  - D ways to break D-cube into (D-1)-cubes, distinct
- Cons:
  - Fragmentation is still possible (easy, even)
  - For D>3 (necessary), cables became ***long***

Degree:	$\log_2 p$
Diameter:	$\log_2 p$
Arc Connectivity:	$\log_2 p$
Bisection width:	$(p \log_2 p)/2$
Cost (links):	$2p$



0-cube, p = 1 | 1-cube, p = 2 | 2-cube, p = 4 | 3-cube, p = 8 | 4-cube, p = 16

Networks: 2D Torus



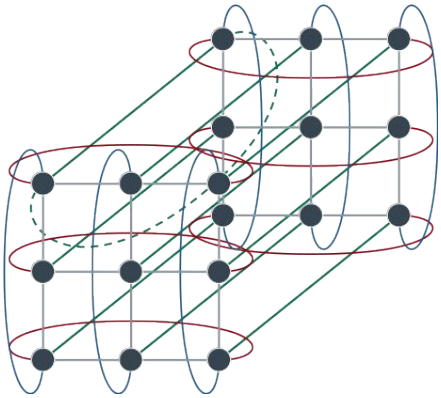
- Every row/column makes a ring
- Pros:
  - Uniform! Easy!
  - Neighbor-neighbor comms are simple
- Cons:
  - Fragmentation is still possible (easy, even)
- Immediate improvement: more dimensions!

Degree:	4
Diameter:	$2 * \lfloor \sqrt{p} / 2 \rfloor$
Arc Connectivity:	4
Bisection width:	$2\sqrt{p}$
Cost (links):	$2p$

Networks: 3D Torus

- Ring of 2D Toroids, q*q*q
- Pros:
  - Many ways to break this into several 2D toroids (even more rings)
  - Great for different types of compute
  - Repeatable, beloved by scientists
- Cons:
  - Many cables, and long
- …More dimensions?

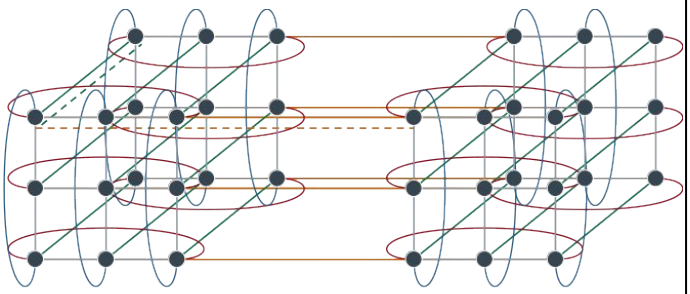
Degree:	6
Diameter:	$3*\lfloor p^{-3} / 2 \rfloor$, or $3*\lfloor q / 2 \rfloor$
Arc Connectivity:	6
Bisection width:	$2\ p^{-6}$, or $2q^2$
Cost (links):	3p



Networks: 4D Torus

- Ring of 3D Toroids, q*q*q*q
- Pros:
  - Many ways to break this into several 3D toroids (even more 2D toroids and even more rings)
  - Great for very specific types of compute (lots of cosmology, physics)
- Cons:
  - Insane networking - with a bit more, can get to 5D (linear in cost)
  - Not a lot of algorithms do well on a 4D lattice
- ***More!***

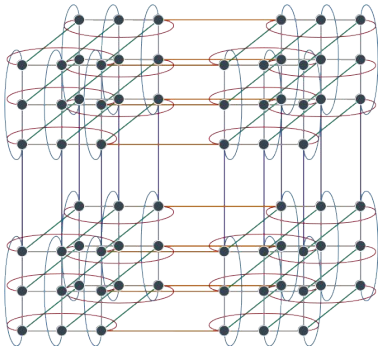
Degree:	8
Diameter:	$4 * \lfloor p^{-4} / 2 \rfloor$, or $4 * \lfloor q / 2 \rfloor$
Arc Connectivity:	8
Bisection width:	$2\ p^{-12}$, or $2q^3$
Cost (links):	4p



Networks: 5D Torus

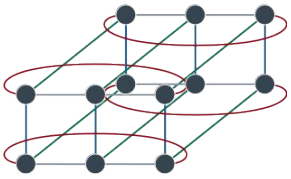
- Ring of 4D Toroids, q*q*q*q*q
- These machines really existed
- Pros:
  - Many, many ways to break this down
  - Many algorithms can be built to communicate in ring, 2D toroid, 3D toroid, and 4D
- Cons:
  - Absolutely massive cabling
  - Switches are necessary and beefy
- But people love these machines
- Alright that's enough.
  - …kind of

Degree:	10
Diameter:	$5*\lfloor p^{-5} / 2 \rfloor$, or $5*\lfloor q / 2 \rfloor$
Arc Connectivity:	10
Bisection width:	$2\ p^{-20}$, or $2q^4$
Cost (links):	5p

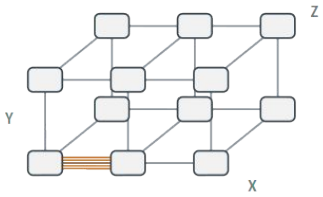


Networks: 6D Toroidial Mesh, or Tofu

- 3D mesh of 3D Toroids
- Want more? Scale the *mesh*.
- Basically: Most of the benefits of 5D toroids minus the awful long cables to make it 6D
- Pros:
  - Still very flexible
  - Still very scalable, even between fixed units
  - Much, *much* less cable
- Cons:
  - Switches and scheduler need to be aware
  - Users really should request node placement according to algorithmic needs, and that's not easy.
- Alright that's *actually* enough.



Fixed unit: 2 × 3 × 2 3D toroid



X × Y × Z mesh/torus of those units

Not quite done. Two more to be complete.

Questions so far?

10 Minute Break

Networks: Vocabulary Short-Hands

- Static: Simple network that won't scale
- Dynamic: Potentially *very* complicated network that will
- Degree: Maximum, how complicated is this network?
- Diameter: Minimum/Average hops, how performant is it?
- Arc connectivity: How easy is it to break it?
- Bisection width: How much bandwidth can it handle?
- Cost (link or switch): How many human headaches and $$ will this cost?

Networks: What we've seen so far (and their issues)

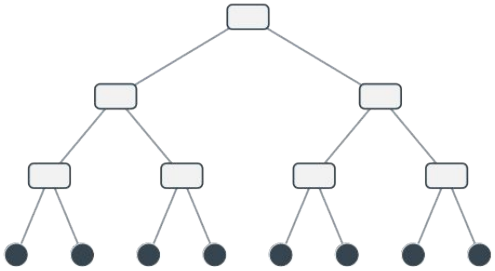
- Line, ring, fully-connected: Too easy
- Bus: The bus cable gets too thick, now the backplane
- Star: The hub gets too pricey
- Crossbar: Switching becomes $$$, now the switch
- Mesh: Too difficult to predict or scale
- Hypercubes: Really cool! (...In theory!)
- Toroids: Really cool! Predictable and configurable. Long cables.

We also need:

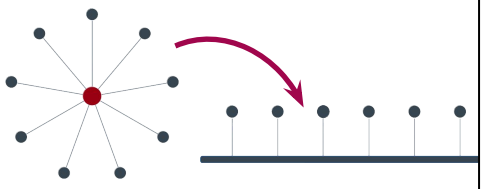
- Trees
- Dragonfly (yes, really)

Networks: Tree

- Really, really affordable performance
- Pros:
  - Not a lot of cables for a short diameter
- Cons:
  - Fragile and bandwidth-limited
- Recall the star to the bus? Same idea, but the bottleneck here is the root's links.

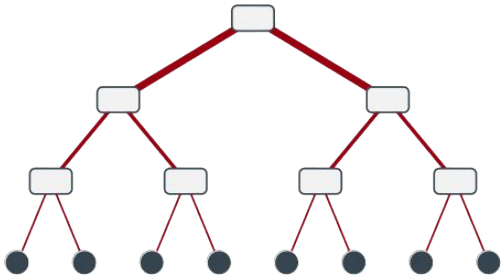


Degree:	1
Diameter:	$2\log_2((p+1)/2)$
Arc Connectivity:	1
Bisection width:	1
Cost (links):	p - 1



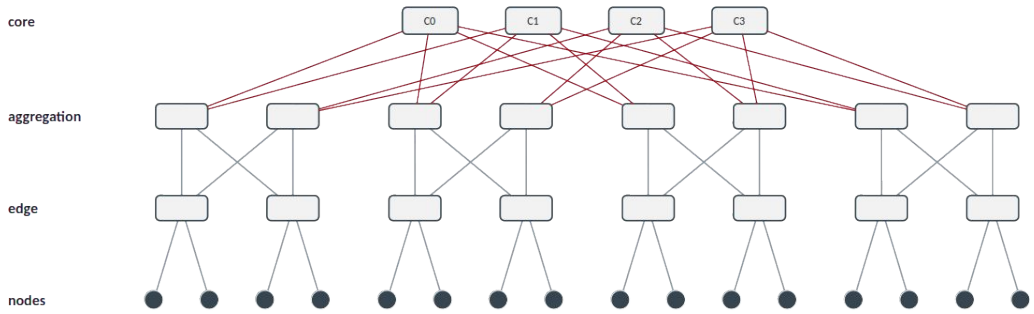
Networks: Fat Tree

- Lieserson (1985): Make the cables "thicker" the closer you are to the root
- Pros:
  - Still not a lot of cables for a short diameter
- Cons:
  - Scaling becomes a problem at enormous degrees



Degree:	1
Diameter:	$2\log_2((p+1)/2)$
Arc Connectivity:	1
Bisection width:	$p/2$
Cost (links):	Depends on how much you want to spend

Networks: Fat Tree (but a real one)

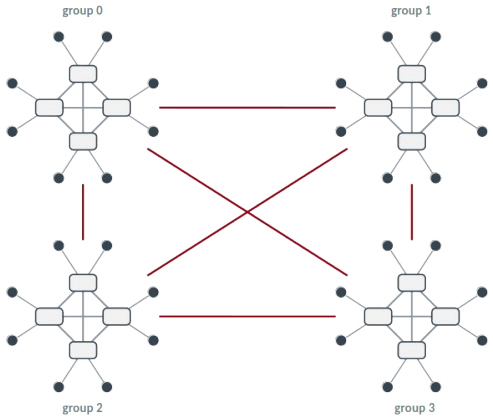


Degree:	1
Diameter:	2n
Arc Connectivity:	1
Bisection width:	p/2
Cost (switches):	$(2n-1)(k/2)^{n-1}$

- Radix k = 4 shown here, k=36 common in production
- $2(k/2)^n$ endpoints permitted
- Ex, two levels = 648 nodes on 54 switches, three levels = 11,664 nodes on 1,620 switches

Networks: Dragonfly

- Take the best (fully-connected, trees, toroids)
- Pros:
  - Works really well (…Most of the time)
  - Highly configurable
- Cons
  - Still a ton of cables, but we aren't escaping this
  - Not at all predictable. Hard on scientists and schedulers alike.
- A best-approximation for a real world system:
  - Easy to set up and switch
  - Dense networking done in copper in cabinets near to compute
  - Group networking done in optical in cabinets at the ends of rows
  - Placement can be a real issue. Fragmentation across groups hurts.



These are complex because Dragonfly is complex. I'll give them to you, but this is a network class discussion and doesn't fit our scope.

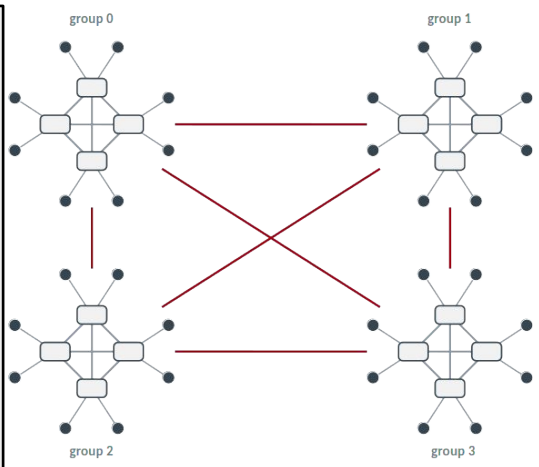
https://pages.cs.wisc.edu/~markhill/restricted/isca08_dragonfly.pdf -- good starting place

Networks: Dragonfly

p = compute nodes per router
a = routers per group
h = global links per router
g = number of groups, max: g = a·h + 1
N = p·a·g compute nodes total

Balanced when a = 2p = 2h

Degree	1 per compute node
Router radix	p + (a−1) + h
Diameter	3 router hops (local, global, local), or 5 links terminal to terminal
Bisection width	$g^2/4$ global links
Arc connectivity	1
Local link cost	g*a(a−1)/2
Global link cost	g(g−1)/2



https://pages.cs.wisc.edu/~markhill/restricted/isca08_dragonfly.pdf -- good starting place

Networks:

- Line, ring, fully-connected: Too easy
- Bus: The bus cable gets too thick, now the backplane
- Star: The hub gets too pricey
- Crossbar: Switching becomes $$$, now the switch
- Mesh: Too difficult to predict or scale
- Hypercubes: Really cool! (...In theory!)
- Toroids: Really cool! Predictable and configurable. Long cables.
- Trees: Very real-world. Good balance of performance.
- Dragonfly: Configurable, scalable, but not predictable.

Network Example: Fat Tree

Connection Machine CM-5 (1993)

Los Alamos National Laboratory
Thinking Machines Corp.

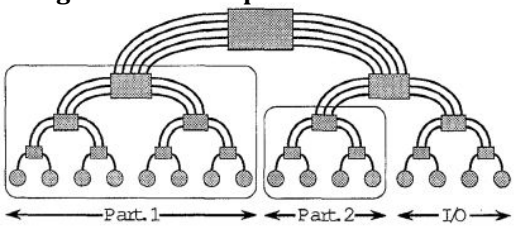


Figure 2: A binary fat-tree. Processors are located at the leaves, and the internal nodes are switches. Unlike an ordinary binary tree, the channel capacities of a fat-tree increase as we ascend from leaves to root. The hierarchical nature of a fat-tree can be exploited to give each user partition a dedicated subnetwork which cannot be interfered with by any other partition's message traffic. The CM-5 data network uses a 4-ary tree instead of a binary tree.

Key specs	
Speed	59.7 GFLOPS LINPACK (131 GFLOPS peak)
Processors	1,024 SPARC nodes, 32 MHz, with vector units
Nodes	1,024 nodes, 32 MB each
Memory	~32 GB total
Interconnect	Fat-tree data network
Power	n/a

https://safari.ethz.ch/architecture/fall2019/lib/exe/fetch.php?media=p272-leiserson.pdf

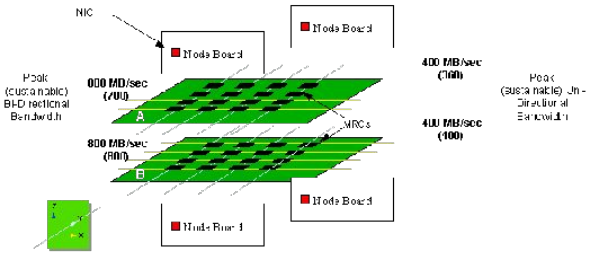
https://www.starringthecomputer.com/appearance.html?f=11&c=15
https://www.threads.com/@dreamwieber/post/DWWgdcwFLE4/the-connection-machines-saga-is-fascinating-you-can-learn-more-about-it-here
https://en.wikipedia.org/wiki/Connection_Machine

Network Example: Split Mesh

Intel ASCI Red (1997)

Sandia National Laboratories, Albuquerque
Built by Intel for DOE

Figure 18 The interconnection topology of the ASCI Red Supercomputer



Key specs	
Speed	1.068 TFLOPS LINPACK (first teraflop)
Processors	~9,200 Intel Pentium Pro, 200 MHz
Nodes	~4,500 compute nodes (2 CPUs each)
Memory	~1.2 TB distributed
Interconnect	38 x 32 x 2 mesh
Power	~850 kW

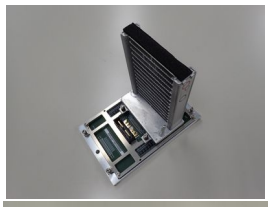
https://cmsdoc.cern.ch/~gutleber/drthesis/node40.html

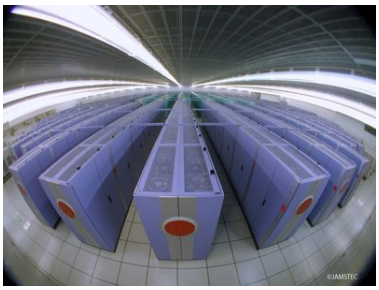
https://en.wikipedia.org/wiki/ASCI_Red
https://www.tomshardware.com/picturestory/696-supercomputer-evolution-over-22-years.html

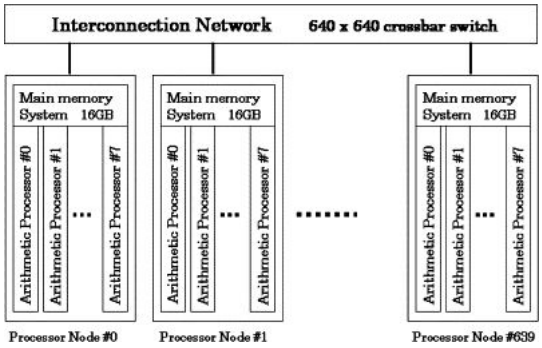
Network Example: Crossbar

Earth Simulator (2002)

JAMSTEC, Yokohama, Japan
Built by NEC







Key specs	
Speed	35.86 TFLOPS LINPACK (40 TF peak)
Processors	5,120 NEC SX-6 vector processors
Nodes	640 nodes, 8 vector CPUs each
Memory	10 TB total
Interconnect	Single-stage crossbar (IXS)
Power	~6 MW

Shinichi Habata, Kazuhiko Umezawa, Mitsuo Yokokawa, Shigemune Kitawaki,
Hardware system of the Earth Simulator,
Parallel Computing,
Volume 30, Issue 12,
2004,
Pages 1287-1313,
ISSN 0167-8191,
https://doi.org/10.1016/j.parco.2004.09.004.
(https://www.sciencedirect.com/science/article/pii/S0167819104001127)
Abstract: The Earth Simulator (ES), developed under the Japanese government's initiative "Earth Simulator project", is a highly parallel vector supercomputer system. In this paper, an overview of ES, its architectural features, hardware technology and the result of performance evaluation are described. In May 2002, the ES was acknowledged to be the most powerful computer in the world: 35.86teraflop/s for the LINPACK HPC benchmark and 26.58teraflop/s for an atmospheric general circulation code (AFES). Such a remarkable performance may be attributed to the following three architectural features; vector processor, shared-memory and high-bandwidth non-blocking interconnection crossbar network. The ES consists of 640 processor nodes (PN) and an interconnection network (IN), which are housed in 320 PN cabinets and 65 IN cabinets. The ES is installed in a specially designed building, 65m long, 50m wide and 17m high. In order to accomplish this advanced system, many kinds of hardware technologies have been developed, such as a high-density and high-frequency LSI, a high-frequency signal transmission, a high-density packaging, and a high-efficiency cooling and power supply system with low noise so as to reduce

whole volume of the ES and total power consumption. For highly parallel processing, a special synchronization means connecting all nodes, Global Barrier Counter (GBC), has been introduced.
Keywords: Supercomputer; HPC (high performance computing); Parallel processing; Shared memory; Distributed memory; Vector processor; Crossbar network

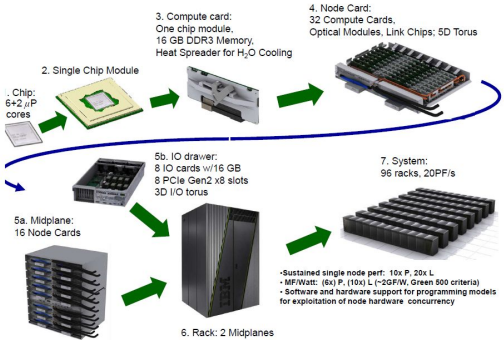
https://museum.ipsj.or.jp/en/heritage/Earth_Simulator.html

https://www.tomshardware.com/picturestory/696-supercomputer-evolution-over-22-years.html

Network Example: 3D Torus

Intrepid (2008)

Argonne Leadership Computing Facility (ALCF)
IBM Blue Gene/P



Key specs	
Speed	450.3 TFLOPS LINPACK (557 TF peak)
Processors	40,960 quad-core PowerPC 450 (850 MHz)
Nodes	40,960 nodes, 163,840 cores
Memory	~80 TB
Interconnect	3-D torus (Blue Gene/P)
Power	~1.3 MW

https://www.sachinpbuzz.com/2014/09/ibm-blue-gene-q-supercomputer.html

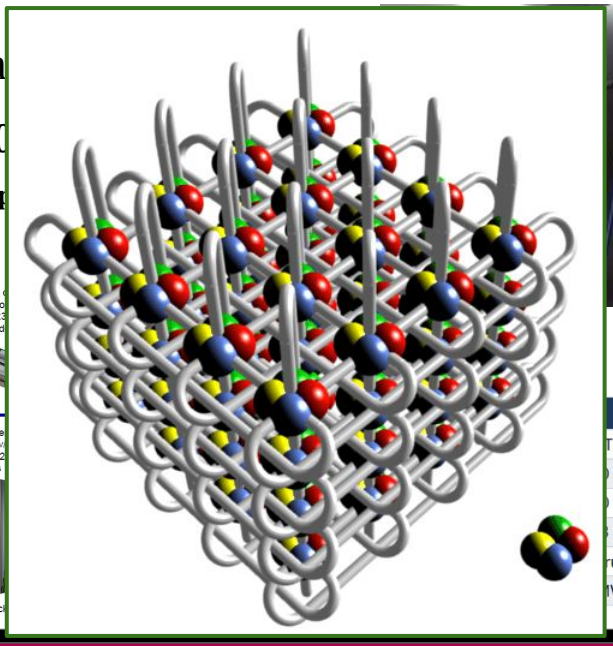
https://www.researchgate.net/figure/Blue-Gene-supercomputer-and-its-torus-network-architecture_fig3_40669949

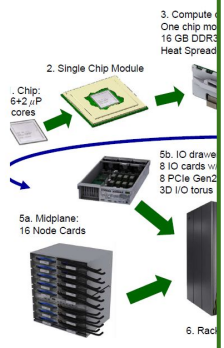
https://news.softpedia.com/news/DOE-Awards-Millions-of-Processing-Hours-149742.shtml

Network Exa

Intrepid (200

Argonne Leadership
IBM Blue Gene/P





TFLOPS LINPACK (557 TF peak)
quad-core PowerPC 450 (850 MHz)
nodes, 163,840 cores
rus (Blue Gene/P)
W

https://www.sachinpbuzz.com/2014/09/ibm-blue-gene-q-supercomputer.html

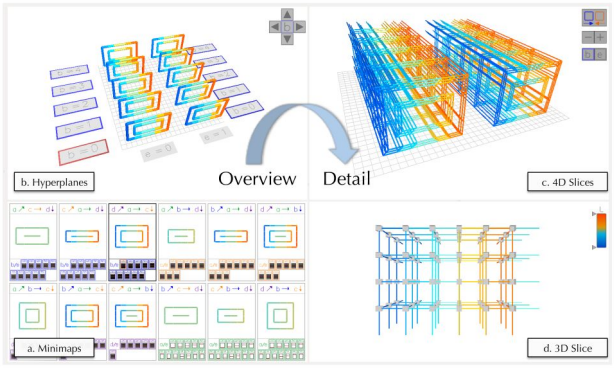
https://www.researchgate.net/figure/Blue-Gene-supercomputer-and-its-torus-network-architecture_fig3_40669949

https://news.softpedia.com/news/DOE-Awards-Millions-of-Processing-Hours-149742.shtml

Network Example: 5D Torus

Mira (2012)

Argonne Leadership Computing Facility (ALCF)
IBM Blue Gene/Q



Key specs	
Speed	8.59 PFLOPS LINPACK (10.06 PF peak)
Processors	49,152 x 16-core PowerPC A2 (1.6 GHz)
Nodes	49,152 nodes, 786,432 cores
Memory	768 TB
Interconnect	5-D torus (Blue Gene/Q)
Power	3.9 MW

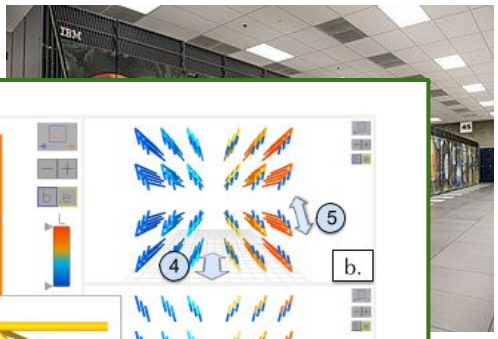
Mccarthy, Collin & Isaacs, Katherine & Bhatele, Abhinav & Bremer, Peer-Timo & Hamann, Bernd. (2014). Visualizing the Five-dimensional Torus Network of the IBM Blue Gene/Q. 24-27. 10.1109/VPA.2014.10.

https://en.wikipedia.org/wiki/Mira_(supercomputer)
https://www.cpushack.com/2013/02/17/ibm-blue-geneq-the-heart-of-a-supercomputer/

Network Example: 5D Torus

Mira (201

Argonne Lead
IBM Blue Gene



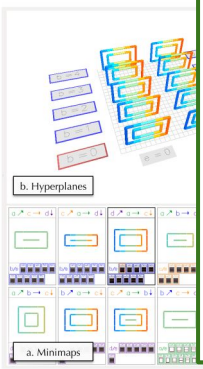


Fig. 4: Link drawing options of the 4D slice inset view: (a) All directions, (b) fourth and fifth dimension, (c) fifth dimension only.

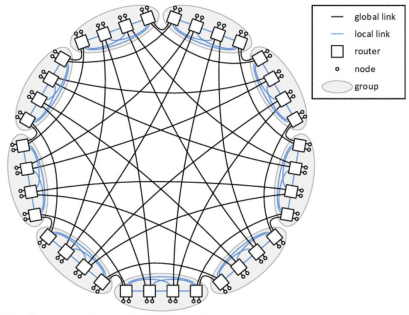
Mccarthy, Collin & Isaacs, Katherine & Bhatele, Abhinav & Bremer, Peer-Timo & Hamann, Bernd. (2014). Visualizing the Five-dimensional Torus Network of the IBM Blue Gene/Q. 24-27. 10.1109/VPA.2014.10.

https://en.wikipedia.org/wiki/Mira_(supercomputer)
https://www.cpushack.com/2013/02/17/ibm-blue-geneq-the-heart-of-a-supercomputer/

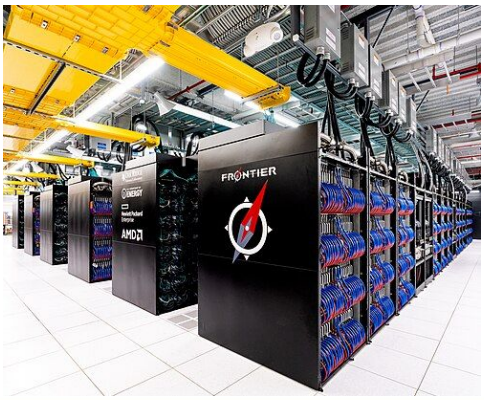
Network Example: Dragonfly

Frontier (2022)

Oak Ridge National Laboratory
HPE Cray EX



https://www.researchgate.net/profile/Enrique-Vallejo-2/publication/261313973/figure/fig2/AS:667782257573894@1536223105142/Sample-Dragonfly-topology-with-h2-p2-a4-36-routers-and-72-compute-nodes.png



Key specs	
Speed	1.102 EFLOPS LINPACK (first exaflop)
Processors	~9,400 AMD EPYC + ~37,000 AMD MI250X GPU
Nodes	~9,400 nodes, ~8.7M cores
Memory	~9.2 PB
Interconnect	HPE Slingshot-11, dragonfly
Power	~21 to 25 MW

https://www.flickr.com/photos/oakridgelab/52280656148/
https://en.wikipedia.org/wiki/Frontier_(supercomputer)
https://mindmatters.ai/2022/06/worlds-fastest-computer-breaks-into-the-exascale/

Questions?

10 Minute Break

How does an HPC system work?

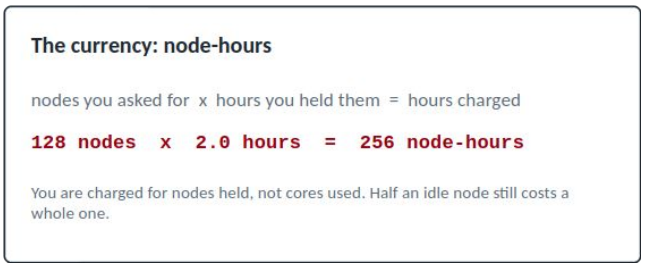
1. Get a really massive machine
2. Give it a huge, high-performance network
3. Get a specialized OS that keeps things safe, sane, and timely, plus a "super-OS" to schedule on it
4. Make a job that can run in parallel
5. Schedule it, requesting whatever resources it needs
6. The scheduler finds a time when the machine has those resources, then sets it to run
7. Job starts, and the nodes do their business
8. Job completes, and the user finds their resulting data

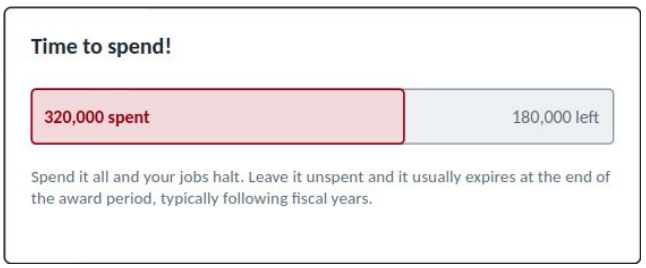
How does an HPC system work?

1. **Get a really massive machine**
2. **Give it a huge, high-performance network**
3. **Get a specialized OS that keeps things safe, sane, and timely, plus a "super-OS" to schedule on it**
4. Make a job that can run in parallel
5. Schedule it, requesting whatever resources it needs
6. The scheduler finds a time when the machine has those resources, then sets it to run
7. Job starts, and the nodes do their business
8. Job completes, and the user finds their resulting data

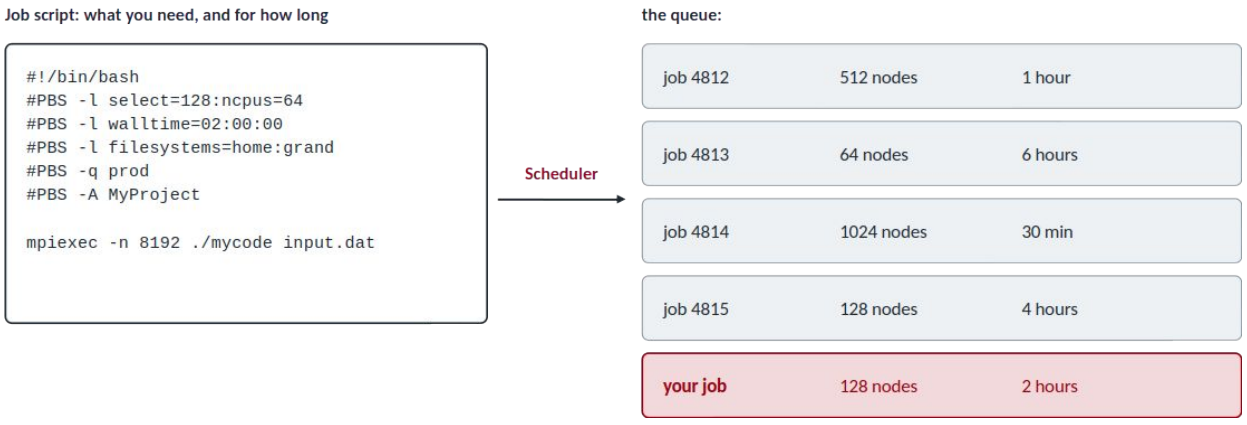
HPC System Conventions:

- Access:
  - Login, often following application/grant proposal
  - Some kind of currency (generally node-hours)
  - User requests runtime via job submission script



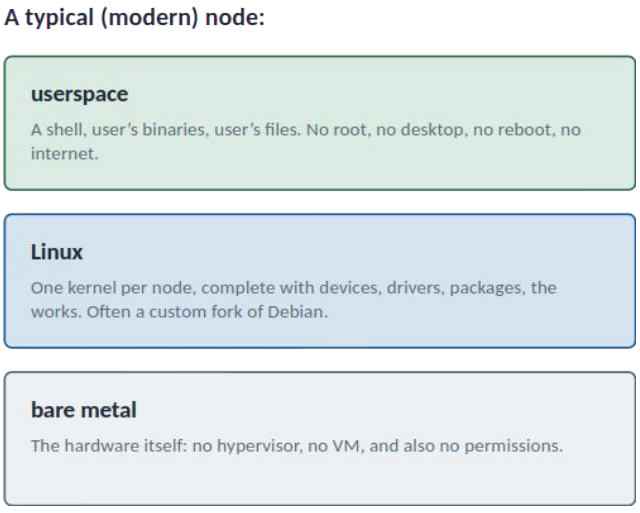


HPC System Conventions:



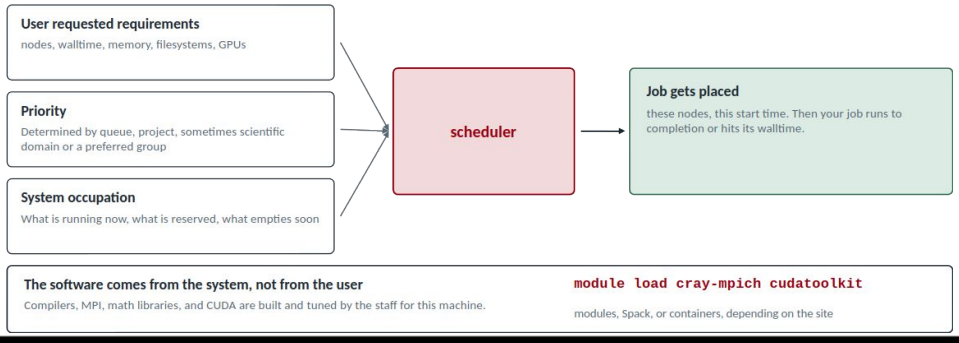
HPC System Conventions:

- Node:
  - Baremetal
  - Linux!
  - Users get userspace (terminal) and nothing else
  - Direct compute node access is limited: usually a login node (which =/= compute node stack)



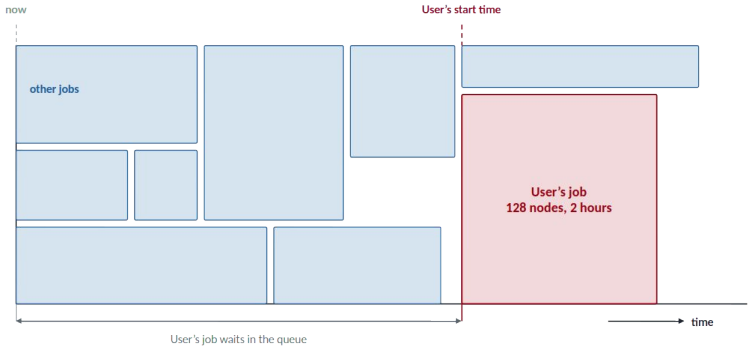
HPC System Conventions:

- Runtime (scheduler):
  - Like processes on an OS
  - An HPC-specific scheduler places jobs according to requirements, priority, and system occupation
  - Packages provided by the system administrators via some mechanism



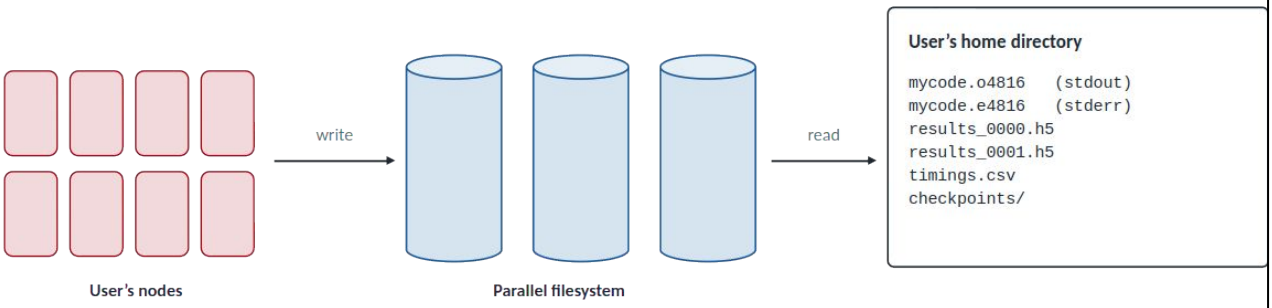
HPC System Conventions:

- Runtime (scheduler):
  - Like processes on an OS
  - An HPC-specific scheduler places jobs according to requirements, priority, and system occupation
  - Packages provided by the system administrators via some mechanism



HPC System Conventions:

- Data:
  - Massive parallel filesystem, typically none on-node
  - Users copy data in, wait for the job to complete, then copy results out



The user also gets a record of the run: nodes used, walltime consumed, and the hours charged to the project.

Example: User/Job Life-Cycle on Aurora (ALCF)

1. User prepares HPC code, using packages available on Aurora
2. User applies for an ALCF account and time on Aurora
    a. Must justify needing a leadership-class machine
    b. Must demonstrate competency using a machine of this scale
    c. Scale, testing, high-demand field, etc.
3. User logs into login node
4. Submits interactive debug job request, 1 hour runtime
    a. Waits, 20 min - 6 hours
    b. Window opens, log in, run subset of job to make sure packages function
    c. Exit
5. Back in login node, submit scaled batch job on debug queue
    a. Submit the job
    b. Wait for completion, usually faster than normal queues but limited scaling and runtime
6. Once complete, check results for correctness
7. Back in login node, submit real jobs in real queues
    a. Backfill: "I don't much care when this runs, charge me fewer currency units"
    b. Preemptable: "My job is fault-tolerant, feel free to interrupt me if a high-priority job comes along, and charge me far fewer currency units"
    c. Standard: "Run my job, please"
    d. High-priority, specialized queues: "We already agreed I get priority access, and now I'm going to use it, so schedule me quickly please"

Example: Scheduler/Job Life-Cycle on Aurora (ALCF)

1. Administrators prepare sets of packages that are well-used and tested for functionality on Aurora nodes
2. PBS Pro, the scheduler, is configured:
   a. FIFO with backfill and preemptability
   b. Queues and priority structures
3. Login nodes are purged for accidental/"accidental" compute
4. PBS receives request for a job schedule, considers:
   a. Number of nodes, user permissions
   b. Priority markers and size determine queue
5. Place job into the queue, populate scheduler metadata
6. When availability opens, place the job
7. When the job exits or its walltime expires:
   a. Update scheduler metadata
   b. Wipe the node

Questions?

How does an HPC system work?

1. Get a really massive machine
2. Give it a huge, high-performance network
3. Get a specialized OS that keeps things safe, sane, and timely, plus a "super-OS" to schedule on it
4. Make a job that can run in parallel
5. Schedule it, requesting whatever resources it needs
6. The scheduler finds a time when the machine has those resources, then sets it to run
7. Job starts, and the nodes do their business
8. Job completes, and the user finds their resulting data

How does an HPC system work?

1. Get a really massive machine
2. Give it a huge, high-performance network
3. Get a specialized OS that keeps things safe, sane, and timely, plus a "super-OS" to schedule on it
4. **Make a job that can run in parallel**
5. Schedule it, requesting whatever resources it needs
6. The scheduler finds a time when the machine has those resources, then sets it to run
7. Job starts, and the nodes do their business
8. Job completes, and the user finds their resulting data

Review

- Networks:
  - Vocabulary
  - Networks
  - Examples
  - Overview of HPC use
- A1: Examples out by Friday, due date adjusted: 26 Sep, 11:59 PM AoE
- Next week:
  - RO2 and HW2 - posted tonight
  - Parallel Programming next week